

Running Phase Explorer — Implementation Report

Summary

Running Phase Explorer is an interactive data visualization dashboard that analyzes a runner's training history from Strava and automatically identifies distinct **training phases** — periods of time when training has a consistent pattern (e.g., steadily building volume, peaking before a race, recovering after a hard block).

The system transforms raw GPS activity data into a structured, navigable story of how training has evolved over time. Instead of looking at hundreds of individual runs, the athlete (or coach) can see the bigger picture: which weeks were a "Building" phase, when did they hit a "Peak", when did volume drop into "Recovery". Phase boundaries are detected automatically using a statistical algorithm, and each phase is labeled based on objective rules about volume and pace.

Technology Stack

Backend — Python

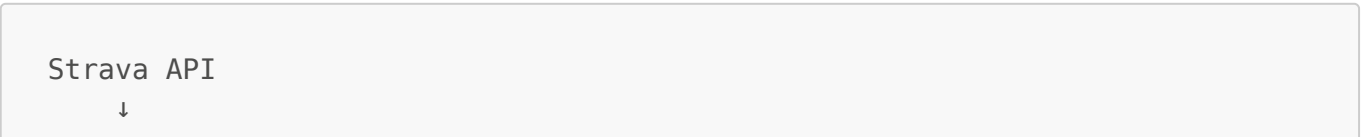
Library	Purpose
Flask	Lightweight web server; serves the frontend and exposes API endpoints
pandas	Weekly data aggregation, time-series manipulation
numpy	Numerical computations (slopes, means, ratios)
ruptures	PELT changepoint detection algorithm
requests	Fetching activity data from the Strava API

Frontend — JavaScript / D3.js

Library	Purpose
D3.js v7	All interactive charts, timelines, heatmaps
Vanilla JS	App state, data loading, view coordination
HTML5 / CSS3	Layout and visual styling

Data Exchange

All data between backend and frontend is passed as **JSON**. The pipeline writes `phases.json` once; the frontend loads it on startup. No database is used.



```
Python pipeline (pandas + ruptures)
  ↓
phases.json
  ↓
D3.js frontend (browser)
```

Section 1 — Overview Timeline

What this section is

The Overview Timeline is the **first thing the user sees**. It shows the entire training history as a single horizontal strip — from the very first recorded run to the most recent one. The goal is to give the athlete a bird's-eye view of all phases across weeks or years.

Elements and what they mean

Phase Strip (color bar at the top)

A continuous horizontal bar divided into colored segments. Each segment = one training phase.

The color immediately tells the user what type of phase that period was — no reading required. Phases are color-coded:

Phase	Color	What it means
Building	Green #4ADE80	Volume is actively growing week-over-week
Peak	Amber #F59E0B	Highest training load; volume is high but plateau
Base	Indigo #818CF8	Moderate, stable training — maintenance period
Recovery	Sky blue #93C5FD	Volume significantly below normal; rest/regeneration
Sharpening	Teal #2DD4BF	Volume decreasing but pace improving (race taper)
Inactive	Gray striped	Gap in training ≥ 10 days; no meaningful runs

Inactive phases are rendered as a **diagonal stripe pattern** (white lines on gray background) to visually signal a "break in the data" rather than a training type.

Volume Chart (area + line below the phase strip)

A time-series area chart showing **km per week** for every week in the dataset. The filled area under the line makes volume drops and spikes easy to spot at a glance.

- X-axis: time (weeks → months → years)
- Y-axis: kilometers per week
- Light horizontal grid lines added for reference

Time Axis

Month and year labels are placed along the bottom. Week-level tick marks show finer granularity. This lets the user orient any phase in calendar time ("that Peak was around October 2024").

Interactive drag-to-zoom

The user can **click and drag** on any part of the Overview Timeline to select a time range. This action loads the Zoom View (Section 2) showing that period in detail.

Section 2 — Zoom / Detail Timeline

What this section is

After the user drags on the overview, this section appears below it and shows a **magnified view** of the selected time window. It offers much richer information per phase and per week than the overview can display.

Elements and what they mean

Phase Strip (detailed version)

Same color-coding as the overview, but now wide enough to show text labels inside each phase block:

- Phase name (e.g., "Building")
- Duration in weeks
- Average km/week for that phase

Clicking a phase block opens the **Heatmap** (Section 3) for that phase.

Metrics Chart

A multi-line chart plotted on top of the zoomed weeks. The user can toggle between three metrics using buttons:

Metric	Unit	What it shows
Pace	min/km	Average running pace per week — lower = faster
Avg HR	bpm	Average heart rate per week — shows exertion level
Efficiency	speed ÷ HR	How much speed the runner gets per heartbeat — higher = more efficient

Efficiency is only shown if at least **40% of runs** in the dataset have heart rate data. Below that threshold the metric is hidden because it would be unreliable.

Distance Bars

Behind the metrics lines, weekly km totals are shown as **vertical bars**. This makes it easy to see volume spikes or drops even without reading the Y-axis number.

Clicking a weekly bar opens the **Week Detail** (Section 4) for that specific week.

Phase Transition Cards

Between consecutive active phases, a small **transition card** appears. It shows what changed between the two phases — in percentage terms:

- Volume change (km/week)
- Pace change
- Efficiency change (if available)

Each card also has a **narrative label** — a human-readable summary of the most significant change (e.g., "volume_drop", "fitness_gain", "frequency_surge"). See the Narrative Algorithm section below for how these are determined.

Section 3 — Weekly Heatmap

What this section is

When the user clicks on a phase in the Zoom View, this section appears. It shows a **grid** where:

- Each **row** = one week within the phase
 - Each **column** = one day of the week (Mon → Sun)
 - Each **cell** = km run on that day (0 if rest day)
-

Elements and what they mean

Cell color

The color of each cell is determined by a **continuous scale**:

- White (**#F3F4F6**) = 0 km (rest day)
- Full phase color (e.g., green for Building) = the highest daily km recorded in that phase

Days with more running appear darker/more saturated. This immediately reveals training patterns — are long runs on Sundays? Is there a mid-week gap? Is the volume spread evenly or concentrated?

The scale is computed per-phase:

```
color = interpolate(white → phaseColor, dailyKm / maxKmInPhase)
```

Hover tooltip

Hovering over any cell shows: week number, total km for that week, average pace.

Section 4 — Week Detail

What this section is

When the user clicks a weekly bar in the Zoom View, this section shows a list of all **individual runs** performed during that week.

Elements and what they mean

Each row in the list represents one run and shows:

Field	Source	Notes
Date	Direct from Strava	Day the run happened
Distance	Direct from Strava (÷ 1000)	Converted from meters to km
Pace	Computed: <code>1000 / speed_ms / 60</code>	Minutes per km
Avg HR	Direct from Strava	Only shown if data available

This is the only view where **raw, unaggregated** data from Strava is displayed. Everything else in the dashboard is aggregated to week level.

Data Calculation — How Metrics Are Computed

Raw data fields (come directly from Strava)

These are stored as-is and not modified:

- `start_date` — ISO timestamp of the run
- `distance` — meters
- `average_speed` — m/s
- `average_heart_rate` — bpm (may be missing)

Weekly aggregation (computed by `features.py`)

All raw runs are grouped by ISO week (Monday–Sunday). For each week:

Metric	Formula
<code>km_total</code>	<code>SUM(distance / 1000)</code> for all runs in the week
<code>run_count</code>	count of runs in the week
<code>avg_pace</code>	<code>MEAN(1000 / speed_ms / 60)</code> — average of per-run paces, in min/km
<code>max_run_km</code>	max single-run distance in the week
<code>avg_run_km</code>	<code>km_total / run_count</code>

Metric	Formula
<code>long_run_ratio</code>	$\text{max_run_km} / \text{avg_run_km}$ — how dominant the longest run is
<code>efficiency</code>	$\text{MEAN}(\text{speed_ms} / \text{heartrate})$ — only when HR is available
<code>km_4w_slope</code>	linear slope of <code>km_total</code> over the last 4 active weeks (OLS)

Empty weeks (zero runs)

Weeks with no runs are filled in to maintain a continuous time series:

Metric	Behavior
<code>km_total</code>	Set to 0
<code>run_count</code>	Set to 0
<code>avg_pace</code>	Set to NaN (no pace to measure on a rest week)
<code>long_run_ratio</code>	Forward-filled from the last non-empty week
<code>efficiency</code>	Forward-filled from the last non-empty week

Pace display conversion

Pace stored internally as `minutes/km` (float). For display, it is converted to `MM:SS` format:

```
display_pace = floor(pace) + ":" + floor((pace % 1) * 60).pad(2)
Example: 5.75 → "5:45"
```

Phase Detection Algorithm

This is the core of the system. The algorithm runs automatically every time new Strava data is loaded.

Step 1 — Inactive Gap Detection (`gaps.py`)

Before detecting phases, the algorithm identifies **long breaks** in training.

Rule: If the gap between two consecutive run dates is ≥ 10 days, that interval is labeled as an **Inactive** phase.

```
For each consecutive pair of run dates (A, B):
    if (B - A) >= 10 days:
        create Inactive phase from A to B
```

The remaining activity is split into **continuous segments** — stretches of training with no long gaps. Each segment is processed independently so that a 3-month gap in 2023 doesn't mix with data from 2024.

Step 2 — Feature Normalization (**normalization.py**)

The PELT algorithm needs all features to be on comparable scales. Raw km/week and raw pace live on completely different numerical scales, so they must be normalized first.

Method: Robust Scaling (Interquartile Range)

```
x_scaled = (x - median(x)) / IQR(x)
where IQR = Q75 - Q25
```

Robust scaling is preferred over standard (z-score) scaling because it is **not affected by outliers** — a single 80 km race week won't distort the entire scale.

Features used for detection:

- **km_total**
- **run_count**
- **avg_pace**
- **avg_run_km**
- **km_4w_slope**
- **efficiency** (only if HR coverage ≥ 40%)

Only **active weeks** (km_total > 0) are included. Rest weeks are excluded from the change detection input to prevent zero-rows from creating false phase boundaries.

Step 3 — PELT Changepoint Detection (**detection.py**)

Algorithm: PELT (Pruned Exact Linear Time)

PELT is a statistical algorithm that finds the locations in a time series where the underlying signal changes most significantly. It solves an optimization problem: find breakpoints that minimize the total within-segment variance, penalized by the number of breakpoints.

Library: **ruptures.Pelt(model="l2")**

The **l2** model minimizes the sum of squared deviations within each segment (i.e., finds where the mean vector of features changes).

Key parameters:

Parameter	Value	Effect
pen (penalty)	5	Controls sensitivity. Higher = fewer, longer phases. Lower = more, shorter phases.

Parameter	Value	Effect
min_size	4 weeks	A phase must be at least 4 weeks long. Shorter candidate segments are rejected.
jump	1	Algorithm checks every week as a candidate breakpoint (no skipping).

Output: A list of week indices where phase transitions occur.

Step 4 — Phase Labeling (`labels.py`)

Once PELT identifies where phases start and end, each phase is labeled using **domain-specific rules** based on computed statistics.

For each phase, two key metrics are computed relative to the athlete's overall training baseline:

```
km_ratio    = mean(phase km/week) / median(all active weeks km/week)
pace_ratio  = mean(phase pace)    / median(all active weeks pace)
slope       = linear trend of km/week within the phase (OLS slope, km per week)
```

The labeling rules are evaluated **in order** (first match wins):

Rule 1 — Peak

```
IF km_ratio >= 1.35 AND slope <= 1.0
→ "Peak"
```

The athlete is training at volume that is **35% or more above their typical level**, and volume is no longer rapidly growing (slope near flat or slightly declining). This represents the high-load block just before a race.

Rule 2 — Building

```
IF slope > 2.0
→ "Building"
```

Volume is increasing at more than **2 km/week per week** — a strong upward trajectory. The athlete is in a progressive overload phase.

Rule 3 — Sharpening


```
IF slope < -1.0 AND pace_ratio < 0.97  
→ "Sharpening"
```

Volume is declining (more than 1 km/week drop) AND the athlete is running **at least 3% faster** than their median pace. This is classic pre-race tapering: less volume, higher quality/speed.

Rule 4 — Recovery

```
IF km_ratio < 0.60  
→ "Recovery"
```

Weekly km is **40% or more below the athlete's typical level**. The body is in a regeneration period.

Rule 5 — Base (default)

```
ELSE  
→ "Base"
```

Everything that doesn't match the above rules is labeled as Base — moderate, stable training that maintains fitness without a specific goal.

Step 5 — Short Phase Merging ([pipeline.py](#))

After labeling, any phase that is **less than 3 weeks long** is merged into its neighboring phase. This prevents noise in the changepoint detection from creating meaningless micro-phases.

The merging only happens within the same continuous training segment — it never bridges across an Inactive gap.

After merging, affected phases are re-labeled and their statistics are recomputed from scratch.

Step 6 — Trend Computation (for display)

Each phase also gets a **trend label** used for small visual indicators in the UI. This is separate from the phase type label and computed as:

```
norm_slope = slope / mean_km_in_phase  
  
if norm_slope > 0.05 → "building"    (slope > 5% of mean weekly km)  
if norm_slope < -0.05 → "tapering"  (slope < -5% of mean weekly km)  
else                  → "stable"
```

This produces a simple up/flat/down arrow indicator shown inside phase cards.

Phase Transition Narratives

At every boundary between two active phases, a transition analysis is computed.

Percent change formula (applied to each metric):

$$\text{pct_change} = ((\text{new_value} - \text{old_value}) / |\text{old_value}|) \times 100$$

Narrative labels are assigned based on which change dominates:

Narrative	Rule
volume_drop	km/week decreased by $\geq 30\%$
volume_surge	km/week increased by $\geq 30\%$
fitness_gain	pace improved by $\geq 5\%$ with volume change $< 20\%$
frequency_drop	runs/week decreased by $\geq 40\%$
frequency_surge	runs/week increased by $\geq 40\%$
pattern_shift	no single metric dominated (everything changed moderately)

The rules are evaluated **in order** — the first matching rule determines the narrative label.

Color Zones — Full Rules

Phase colors

Each phase type has a fixed background color and a matching dark text color for contrast:

Phase	Background	Text (for contrast)
Building	#4ADE80 (bright green)	#14532D (dark green)
Peak	#F59E0B (amber)	#78350F (dark brown)
Base	#818CF8 (indigo)	#312E81 (dark indigo)
Recovery	#93C5FD (sky blue)	#1E3A5F (dark navy)
Sharpening	#2DD4BF (teal)	#134E4A (dark teal)
Inactive	#B0B7C3 (gray) + diagonal stripe pattern	—

The diagonal stripe on Inactive phases is an SVG `<pattern>` element — a visual convention for "no data / break" that avoids implying this was a deliberate training type.

Heatmap cell colors

Each cell in the heatmap is colored on a **continuous scale** from white to the phase color:

```
color = d3.interpolate("#F3F4F6", phaseColor)(dailyKm / maxKmInPhase)
```

- A rest day (0 km) → white
- The day with the highest km in the phase → full phase color
- Everything in between → proportional shade

This makes it easy to see which days are "heavy" vs "light" within a phase without reading numbers.

Transition card metric colors

In the Phase Transition Cards, each metric change is colored green or red based on whether the change is **good or bad for the athlete**:

Metric	"Good" direction	Color if good	Color if bad
km/week	Up (more training)	Green #3B6D11	Red-brown #993C1D
avg pace	Down (faster)	Green	Red-brown
efficiency	Up (more efficient)	Green	Red-brown
runs/week	Depends on context	Green	Red-brown

The logic: "bigger is better" for volume and efficiency; "smaller is better" for pace (faster = lower number).

Configuration Reference

All key thresholds are centralized in `backend/config.py`:

Constant	Value	What it controls
INACTIVE_GAP_DAYS	10	Days of gap before marking Inactive
MIN_PHASE_WEEKS	4	Minimum phase length for PELT
PELT_PENALTY	5	Changepoint sensitivity
PELT_JUMP	1	Week resolution for PELT
HR_MIN_COVERAGE	0.40	Min fraction of runs with HR data
MIN_MERGE_WEEKS	3	Short phases smaller than this are merged
Peak km_ratio	≥ 1.35	35% above median volume
Building slope	> 2.0 km/w	Strong weekly growth

Constant	Value	What it controls
Sharpening slope	< -1.0 km/w	Volume decline
Sharpening pace_ratio	< 0.97	3% faster than median
Recovery km_ratio	< 0.60	40% below median volume
Trend boundary	$\pm 0.05 \times \text{mean km}$	building / stable / tapering arrow
Volume surge	$\geq +30\%$	Narrative threshold
Volume drop	$\leq -30\%$	Narrative threshold
Fitness gain pace	$\leq -5\%$	Narrative threshold
Frequency change	$\geq \pm 40\%$	Narrative threshold

Project File Structure

```
infVizProject/
├── backend/
│   ├── app.py           # Flask server + API routes
│   ├── pipeline.py      # Main orchestration: gap → segment → detect →
│   │   │   label → merge
│   ├── config.py        # All constants and thresholds
│   ├── detection.py     # PELT wrapper (ruptures)
│   ├── labels.py        # Phase labeling rules
│   ├── features.py      # Weekly aggregation from raw runs
│   ├── gaps.py          # Inactive gap detection
│   ├── segments.py      # Segment splitting on gaps
│   ├── normalization.py # Robust scaling (IQR-based)
│   └── utils.py         # Logging helpers
├── frontend/
│   ├── index.html       # Single-page app entry point
│   ├── js/
│   │   ├── main.js      # App init, Strava sync button
│   │   ├── dataLoader.js # Load phases.json from disk
│   │   ├── colors.js    # Phase color definitions
│   │   └── state.js      # Global app state (selected phase, week, zoom
│   │   │   range)
│   │   ├── utils.js     # Formatting helpers (pace, dates, percentages)
│   │   └── views/
│   │       ├── overview.js # Full-history timeline + drag-to-zoom
│   │       └── zoomTimeline.js # Detailed phase view + metrics chart + week
│   │   │   bars
│   │       ├── heatmap.js # Weekly grid heatmap per phase
│   │       ├── weekDetail.js # Individual run list for a selected week
│   │       ├── breakpoints.js # Phase transition cards
│   │       └── efficiency.js # Efficiency sub-chart (when HR available)
│   └── css/
```

```
├── main.css
├── timeline.css
└── detail.css

├── data/
│   └── raw_activities.json    # Raw Strava export
├── output/
│   └── phases.json           # Pipeline output consumed by frontend
```